ME2605 Training Neural Networks: Theory and Practice @ SLAI

co-list: DDA6204 Selected Topics-Training Neural Networks: Theory and Practice @ CUHK-SZ

Lecture 1.2: Overview of Training Theory

Ruoyu Sun

Today’s Plan

1 **Review** — ML basics: from function fitting to loss

2 **Online Quiz 1** — ML basics · 10 min

3 **The Three Errors** — where does test error come from?

4 **Live Demo** — watch the three errors happen

5 **Diagnosis** — how to tell them apart

6 **Online Quiz 2** — the three errors · 10 min ·

A Thought Experiment: Back to 2005

You are back in 2005.

No code survives the trip. No papers either.

What do you need to make neural nets work?

Think for a minute — answers on the next pages.

Why DL Failed Before 2005: Hinton's Four Reasons

- Our labeled **datasets** were thousands of times too small.
- Our **computers** were millions of times too slow.
- We **initialized** the weights in a stupid way.
- We used the wrong type of **activation**.

Data and compute were missing — and so were the right training tricks.

The Time-Traveler’s Answers

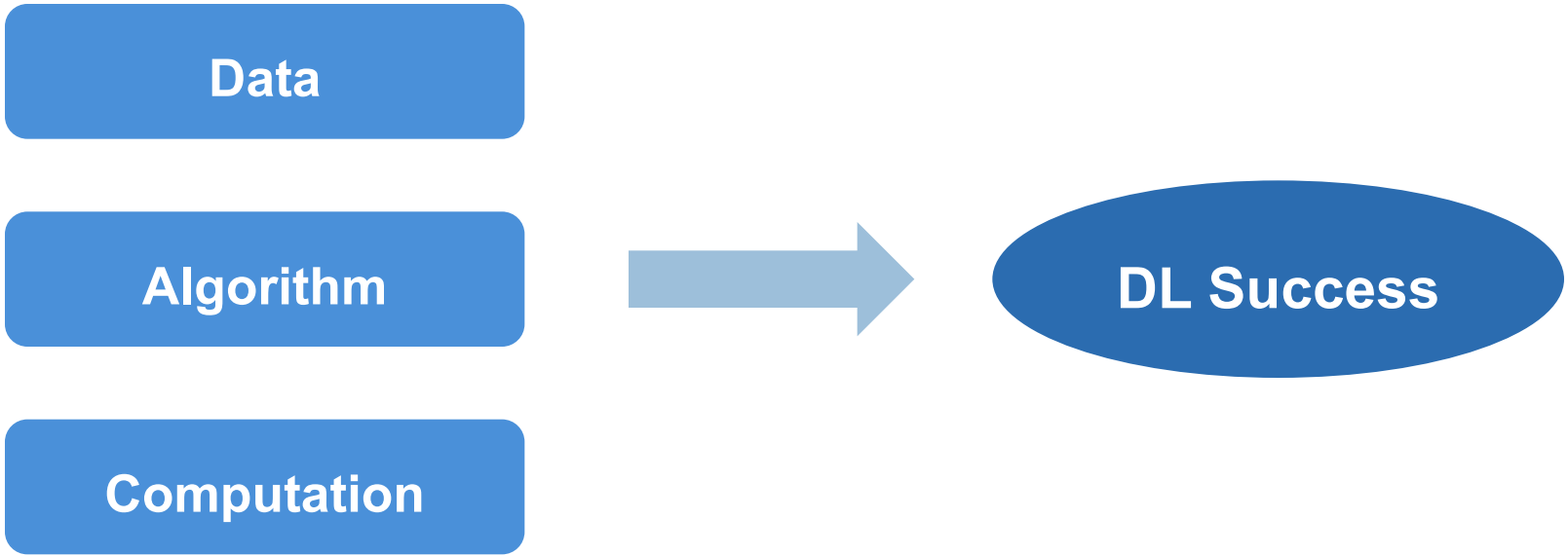
1) **Compute matters** — bet on GPUs early.

2) **Data matters** — build labeled datasets; or collect text data.

3) **Training framework matters**—architecture, algorithms, tricks (deep & wide CNNs or transformers, Adam, LayerNorm, etc.).

We do not discuss how to make GPUs.
We discuss the training framework.

The Recipe of the Deep-Learning Revolution



Next: Re-answer this question, from a theory perspective

SECTION 1

ML Basics Review

function fitting & loss — needed for Quiz 1 and for the whole course

Supervised Learning (SL)

Goal: learn a mapping from inputs to output.

Example: is this image a cat or a dog?
A mapping f: image $\rightarrow$ label $\in$ {cat, dog}

The “naive” way: hand-crafted “features” — face shape, ear size, etc.

This dominated vision for decades — but it was brittle and hard.

Basic SL Formulation

Consider a supervised learning problem with n samples (x_i, y_i), i=1,2,…,n

- A neural network: $f_\theta(x)$
- It makes prediction on each x_i, and we compute a loss $l(f_\theta(x_i), y_i)$, which shall be small when $f_\theta(x_i)$ is close to y_i

- **Training stage**: solve an optimization problem

$$(P1) \quad \min_\theta F(\theta) \triangleq \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f_\theta(x_i))$$

i.e., find good neural-net parameter to minimize the empirical loss

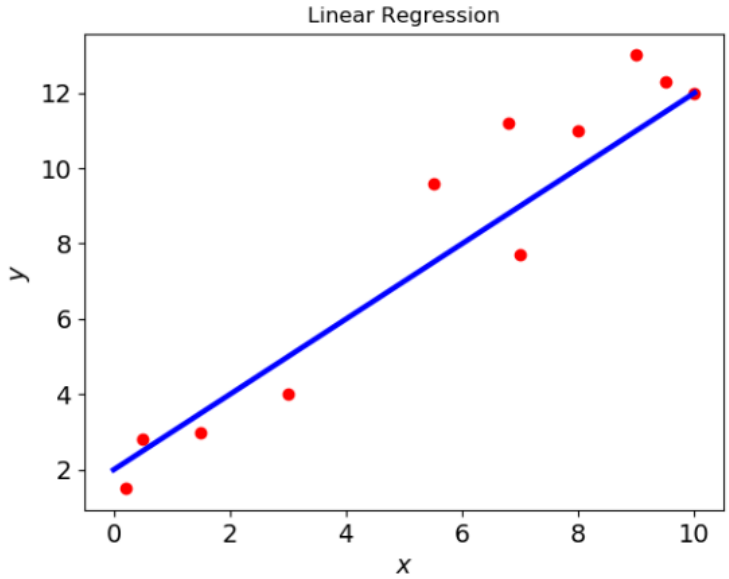
Inference stage: use the learned f to label new, unseen data.

Supervised Learning

Example: classify cat vs dog.

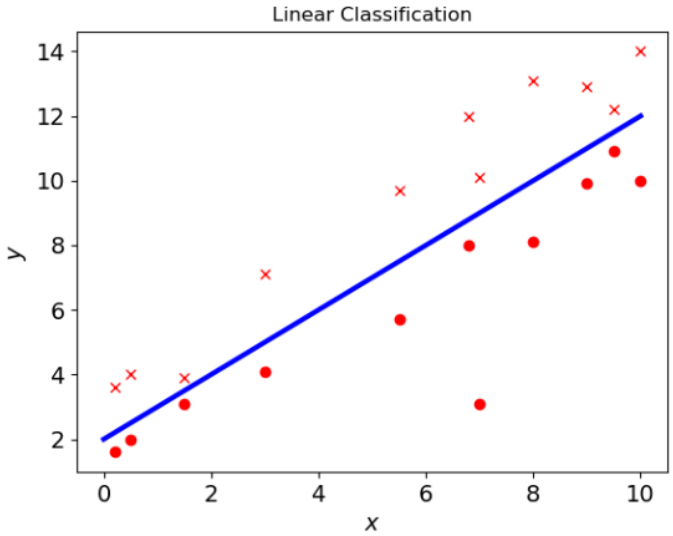
- Data: 1000 cats + 1000 dogs, with correct labels.
- **Train**: tune θ so that $f_\theta(x)$ matches the true label on every training image.
- **Inference**: apply $f_\theta(x)$ to new images, to tell it is a dog or cat.

Example: Linear Regression



- Model: $f_\theta(x)$ = w·x + b, with scalar parameters w and b.
- Parameter: $\theta = (w, b)$

Example: Linear Classification



- Model: $f_\theta(x)$ = sign(w·x + b), scalar w and b; labels in {+1, −1}.
- Parameter: $\theta = (w, b)$

From Distance to Logistic Loss

Classification: (x_i, y_i), $i = 1, \ldots, n$, where $y_i \in \{+1, -1\}$.

- We want w that makes $\text{dist}(w^T x_i, y_i)$ small.
- A standard choice of "dist": the **logistic loss** $\log(1 + \exp(-y\hat{y}))$.

$$\min_w \sum_i \text{dist}(w^T x_i, y_i) = \sum_i \log(1 + \exp(-y_i w^T x_i)).$$

Why this design: smooth (differentiable) · convex · penalizing wrong answers.

Error vs Loss

Error = the 0/1 misclassification rate — the goal, but not differentiable.

Loss = a smooth surrogate we actually minimize — GD needs smoothness.

Hope: loss ↓ ⇒ error ↓.

Disclaimer: Sometimes, we do not distinguish ”training error” and “training loss”.

Sigmoid & Binary Cross-Entropy

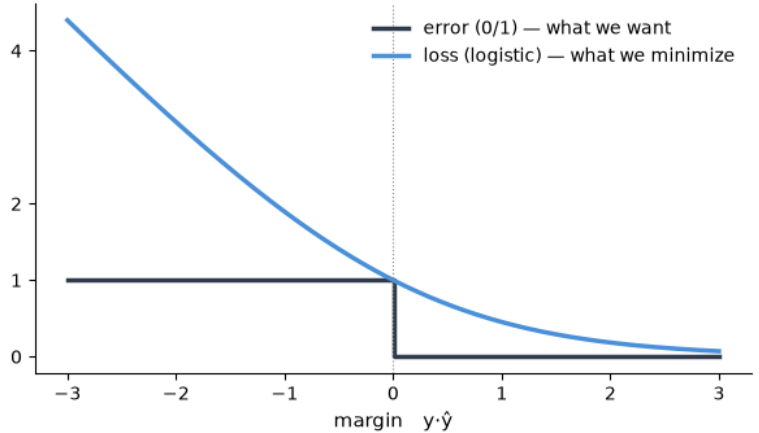
Example: cat vs dog.

Ideal loss: sign function.

--predict the same sign: correct, error 0;

--predict wrong sign: wrong, error 1

Surrogate loss: $\sigma(z) = 1 / (1 + e^{-z})$



Probability view: predict probablity $\sigma(z) = P(\text{dog} \mid x)$.

Minimizing the logistic loss = maximizing the likelihood. Numbers: $\sigma(2) \approx 0.88$, $\sigma(-1) \approx 0.27$

Quiz staple: given z, compute $\sigma(z)$; z = 0, output 0.5; any z, output in (0,1).

Multi-class: Softmax & Cross-Entropy (single sample)

K classes → prediction: K scores: $a = (a_1, \ldots, a_K)$.

prediction probability p = softmax(a), i.e.,

$$p_k = \exp(a_k) / \sum_{j=1}^{K} \exp(a_j)$$

True class k^*, *then*

$$\textbf{loss } L(a) = -\log p_{k^*}$$

Gradient expression:

$$\partial L/\partial a = p - \text{one-hot}(k^*)$$

gradient = prediction − one-hot truth

e.g.

K = 2, prediction (0.5, 0.5),

gradient = (0.5, 0.5) - (0, 1)

Example 1: K = 2; predict (0.5, 0.5), then L = ln 2 ≈ 0.693

Example 2: K = 5; prediction p_{k^*} = 0.7 , then L = −ln 0.7 ≈ 0.357

Quiz staples: softmax([0, 0]) = [0.5, 0.5]; uniform prediction, 2 class, loss = ln 2; 3 class, loss = ln 3.

Big O Notation

For scalar $f(x)$ and positive $g(x)$, compare their growth as $x \to \infty$.

$f = O(g)$: $\limsup_{x\to\infty} \frac{|f(x)|}{g(x)} < \infty$

$f = \Omega(g)$: $\liminf_{x\to\infty} \frac{|f(x)|}{g(x)} > 0$

$f = \Theta(g)$: $f = O(g)$ and $f = \Omega(g)$

$f = o(g)$: $\lim_{x\to\infty} \frac{f(x)}{g(x)} = 0$

Examples as $n \to \infty$

$n^3 + n + 2 = \Theta(n^3)$

Also $O(n^4)$ and $o(n^4)$;

also $\Omega(1)$, $\Omega(n^2)$ and $\Omega(n^3)$.

$\frac{1+n}{n^3} = \Theta(1/n^2)$

Also $O(1/n)$ and $\Omega(1/n^3)$.

At a finite limit $x \to a$, use the same bounded-ratio test: $\varepsilon^2 + \varepsilon^3 = O(\varepsilon^2)$ as $\varepsilon \to 0$.

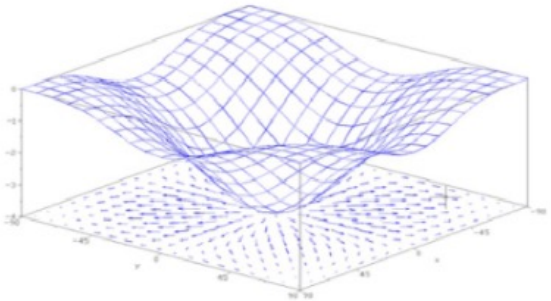
Gradient

Suppose $f: \mathbb{R}^n \to \mathbb{R}$ is a differentiable function.

e_i: the ith coordinate unit vector. The partial derivative is

$$\frac{\partial f(x)}{\partial x_i} = \lim_{t \to 0} \frac{f(x + te_i) - f(x)}{t}$$

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n} \right)^T \in \mathbb{R}^{n \times 1}$$



The gradient points toward the fastest increase per unit Euclidean distance. Its norm is that rate.

Hessian

Suppose $f: \mathbb{R}^n \to \mathbb{R}$ is a twice differentiable function.

Hessian: $H(x) = \nabla^2 f(x), \qquad H_{ij}(x) = \frac{\partial^2 f(x)}{\partial x_i \, \partial x_j}, \quad i, j = 1, \ldots, n.$

$H(x) \in \mathbb{R}^{n \times n}$ records local curvature. Let $h = y - x$ be a small displacement.

$$f(x + h) = f(x) + \nabla f(x)^T h + \frac{1}{2} h^T H(x) h + o(|h|_2^2)$$

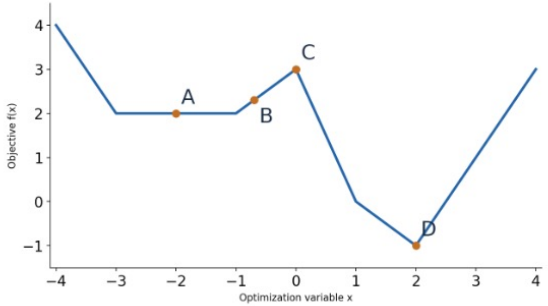
Local Minimum and Maximum

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & x \in \mathbb{R}^n \end{array}$$

- **Objective function** $f : \mathbb{R}^n \to \mathbb{R}$ is a continuous function
- **Optimization variable** $x \in \mathbb{R}^n$
- **(Unconstrained) local minimum** $\hat{x}$: $\exists \epsilon > 0$ s.t. $f(x) \geq f(\hat{x})$, for all $\|x - \hat{x}\| \leq \epsilon$;
  i.e., x^* is the best in a small enough neighborhood
- **(Unconstrained) global minimum** x^*: $f(x) \geq f(x^*)$ for all $x \in \mathbb{R}^n$
- Strict global minimum: change $f(x) \geq f(x^*)$ to $f(x) > f(x^*)$ in the above definition. Similar for "strict local minimum"
- Switching the direction of inequalities, we obtain "local maximum", "global maximum", etc.

Local Minimum and Global Minimum

Question: Which are local minima, which are global minima, and which are strict local minima?



Possible confusion of "local minimum": some people may think A is NOT local-min, and only D is local-min.

- This is because their notion of "local-min" should actually called "strict local-min"

Conditions for Local Min

Necessary at a local minimum

$$\nabla f(x^*) = 0, \qquad H \succcurlyeq 0$$

Sufficient for a strict local minimum

$$\nabla f(x^*) = 0, \qquad H \succ 0$$

$H \succcurlyeq 0$: positive semidefinite, meaning $v^T H v \geq 0$ for every vector v.

$H \succ 0$: positive definite, meaning $v^T H v > 0$ for every nonzero vector v.

A stationary point has zero gradient. Necessary conditions alone do not certify a local minimum.

Gradient Descent Method

Hill climbing: try nearby directions;

choose the largest increase in height.

For minimization, move downhill:

$$x^{r+1} = x^r - \alpha \nabla f(x^r)$$

x^r: the current iterate; r counts updates.

$\alpha > 0$: learning rate (lr in the demo).

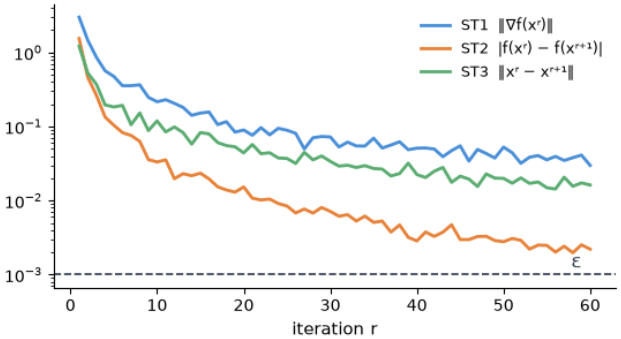
$-\nabla f$: opposite to steepest increase.

The same local slope idea gives ascent (+) or descent (−), depending on the objective.

Stopping Criteria

When should training stop? One of the following criteria:

- **ST1:** $\|\nabla f(x^r)\| \le \varepsilon$
- **ST2:** $|f(x^r) - f(x^{r+1})| \le \varepsilon$
- **ST3:** $\|x^r - x^{r+1}\| \le \varepsilon$



- **ST4:** **r =** MaxIter (i.e. reaches maximal iteration)

For GD with constant stepsize, ST3 is equivalent to ST1; in general, can be different.

Typical choice of ε: $\varepsilon = 1\times10^{-3}$ (low precision) to $\varepsilon = 1\times10^{-10}$ (high precision).

Which one is the best?

1D Logistic Regression: Objective Function

Problem setup: two data points in $\mathbb{R}$, x_1 and x_2; labels $y_1 = 1, y_2 = -1$.

For simplicity, classify with wx_i (drop the bias b for now).

Objective function:

$$f(w) = \tfrac{1}{2}\log(1 + \exp(-wx_1)) + \tfrac{1}{2}\log(1 + \exp(wx_2)).$$

Initialization: $w_0 = 5$ (up to your choice)

GD Algorithm: $w_{r+1} = w_r - \alpha\nabla f(w_r)$.

Train Error, Test Error, Overfitting

Train error = error on the training set.

Test error = error on test data.

Overfitting: train error is small, while test error stays large.

Next section: a numeric example — then the three-error decomposition.

Quiz staple: know the difference between train error and test error.

Training Error vs. Test Error

Suppose you train a classifier: cat or dog?

• Given: **1,000 cat images + 1,000 dog images**, all with correct labels.

• You train a neural net, get all 1,000 cats are correct; **820** dogs predicted “dog”, **180** dogs predicted “cat”.

Training error = 180 / 2,000 = **9%**

• What we really care about is **test error** —

we hold out another 500 cats + 500 dogs, never used in training.

Say the net makes 150 mistakes on this held-out set: **test error = 15%**.

QUIZ 1

ML Basics

10 min · multiple-choice questions · covers Section 1

Step 1: Click the quiz website:
http://47.238.4.125:3100

Step 2: 课堂代码: UNCBXZ

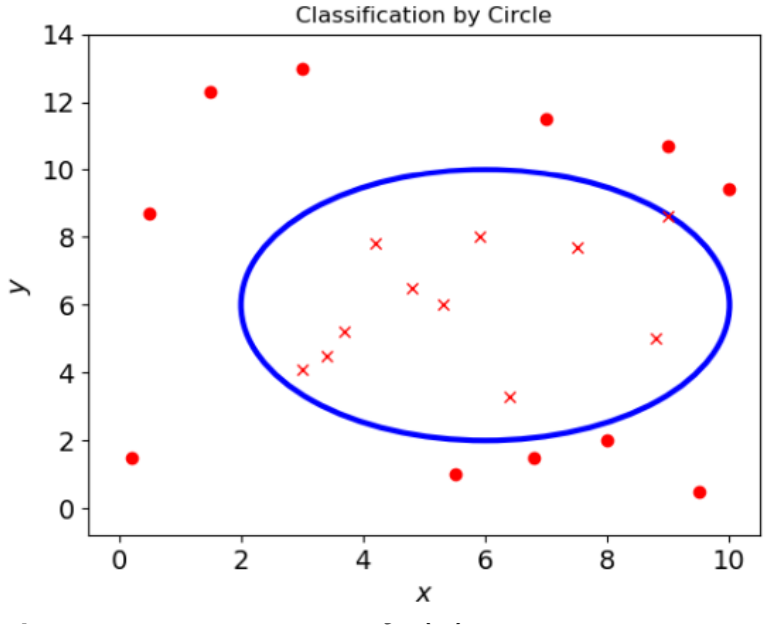
Step 3: Enter your 学号 & 姓名.

Step 4: Start answering quiz questions.

SECTION 2

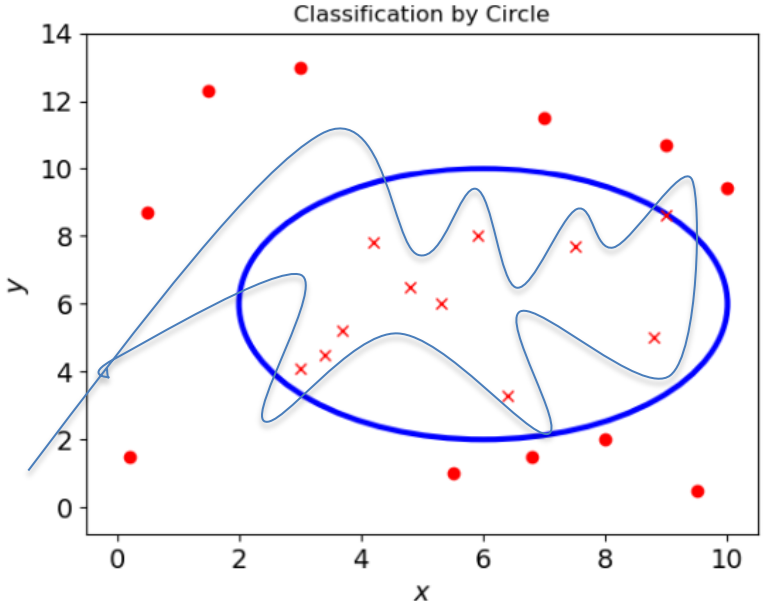
Three-Part Decomposition of Test Error

Example: Circle Classification



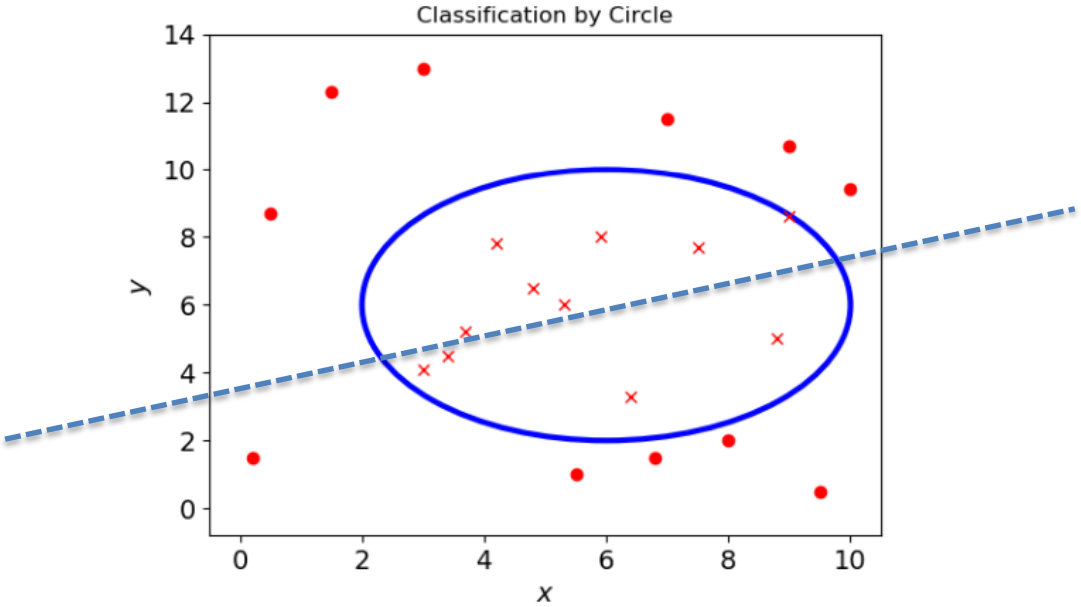
- How should we
- Set of ellipsoid-classifiers

Generalization and Overfitting



- What if we fit a 100th-order polynomial?
- Overfitting; or: large generalization gap

Representation Power



- What if we use a linear classifier to learn?
- We say: the model lacks representation power.

Training v.s. Generalization

Back to the cat/dog classifier: what matters is **test error** — say 15%.

First split: training error is 9%, so the generalization gap is 6%.

Test error = training error + generalization gap

15% = 9% + 6%

Representation v.s. Optimization

Imagine 1000000 GPUs searching the solution for 1000000 centuries:

the best achievable error might be 2% — that is the representation error.

The remaining 7% is the optimization error.

Training error = representation error + optimization error

9% = 2% + 7%

Test error 15% =

2% representation error + 7% optimization error + 6% generalization gap

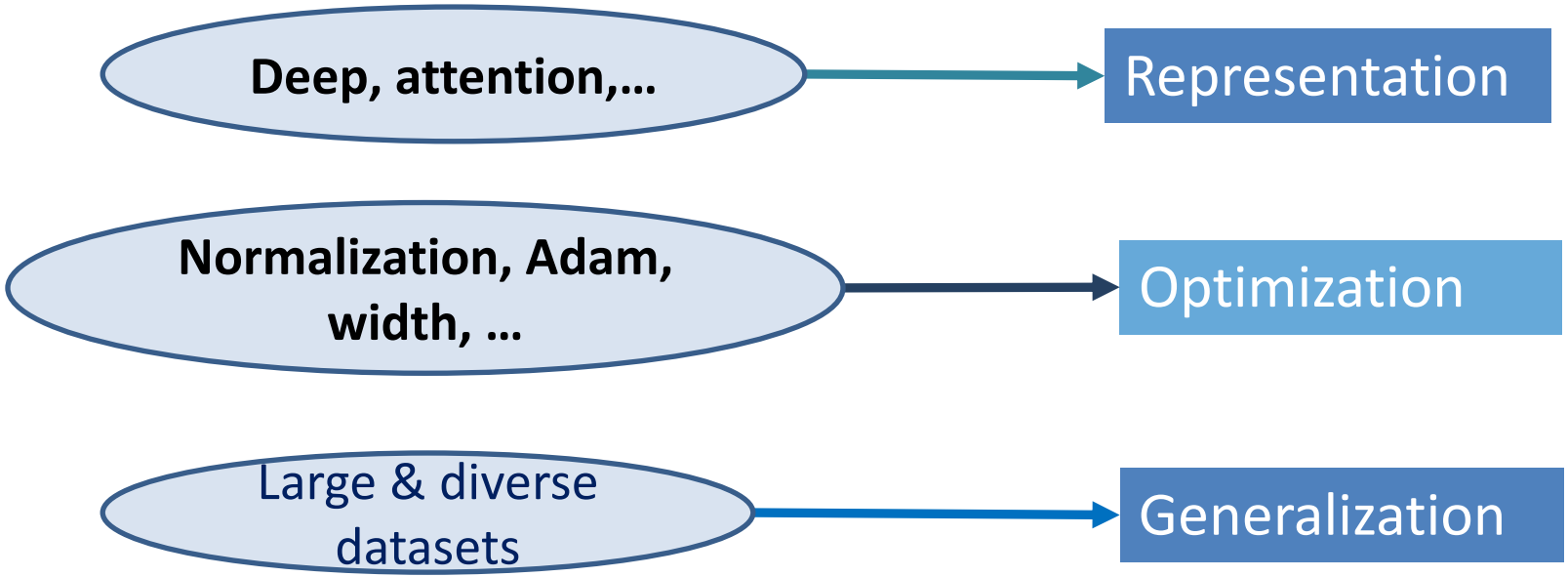
Three-Error Decomposition

1. **Representation:** Can a neural network express the target function?
2. **Optimization:** Can training find parameters with small empirical loss?
3. **Generalization:** Will the trained model perform well on unseen data?

Test error = representation error + optimization error + generalization gap

Training error = representation error + optimization error

Key Ingredients of Good NN



Huge Models Are Common

- Among all the tricks, one dominates: **“use a large model”**

- ResNet-50: 25 million parameters.

GPT3 in 2022: 175m parameters.

Deepseekv4 pro in 2026: 3T (3 万亿) parameters.

GPT Bel to come: guess ~10T

Why so huge?

Representation & optimization.

Why Learn Theory Decomposition?

Scenario 1 (frontier lab):
Deepseek/openAI wants to improve LLM.
Which direction to improve?
--more & diverse data?
--bigger model?
--new architecture?
--**Efficiency**: faster & low-memory algorithm?

Optimization? **Generalization?**

Representation?

e.g. coding data first, why?
e.g. towards 10T, why?
e.g. loopy transformer, why?
e.g. muon, why?

Scenario 2: Use NN to predict RN structure.
Shall we use Transformers? What size? What algorithm?

Boss needs to decide!

What Do They Look Like in Real Training?

The three errors seem to be theoretical.

What do they look like in practice?

→ **First, discuss how to compute them;**

Then, watch them in a live demo.

SECTION 3A · DIAGNOSIS

Generalization Gap

works on train, fails on test?

Train/Test Gap

Generalization (泛化): Core of ML;
extremely easy & extremely hard.

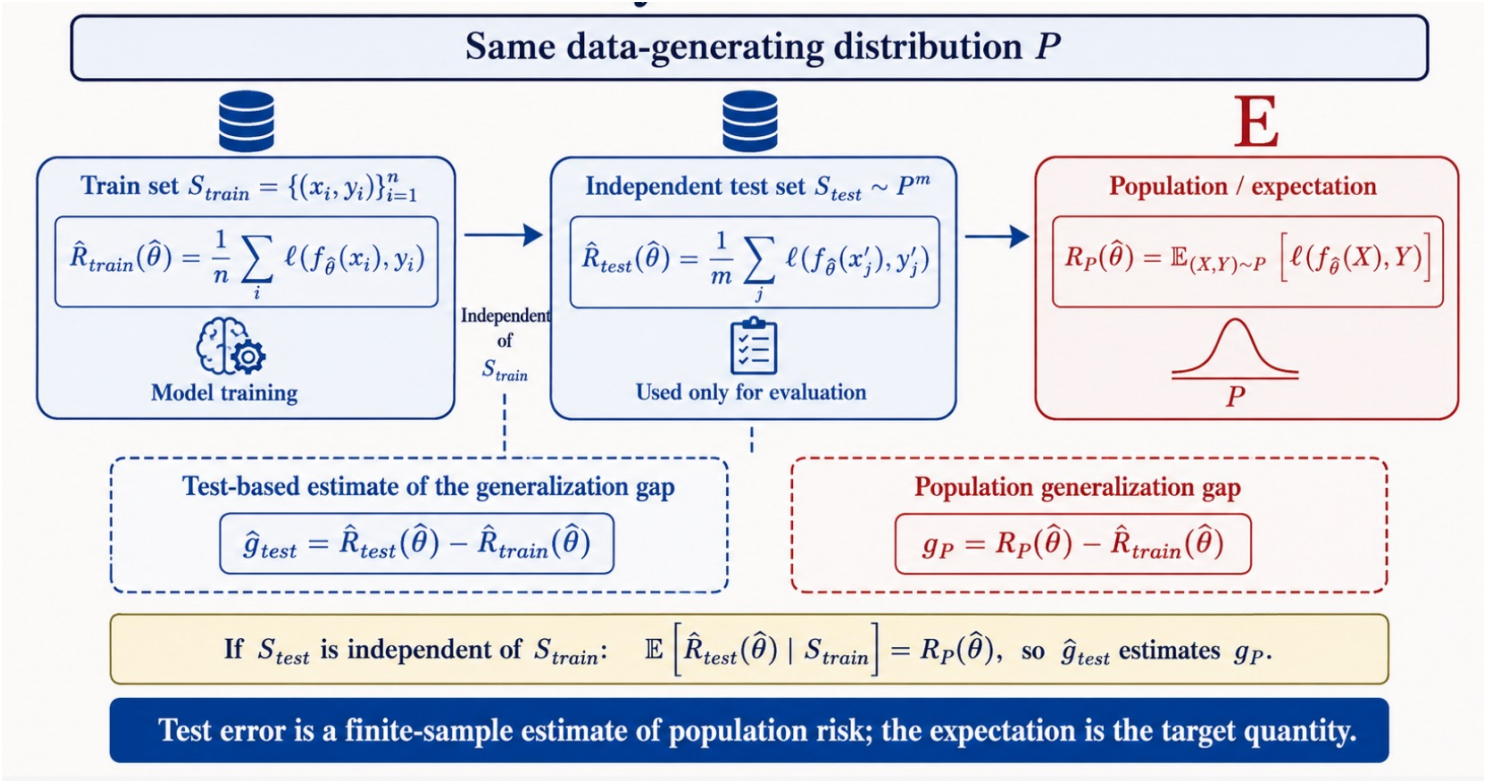
- Seem easy to compute: Train error low, test error high

  → the gap is the generalization gap.

Caveat 1: What we really care, is the error on the "distribution", i.e., on "ALL" samples.
In practice, you always pick a test data set; not the whole data distribution.

Implication 1: Gap between "benchmark" & "user feeling". ["刷榜"]
Implication 2: Performance on two test datasets can be different.

Generalization Gap: Two Forms



What Distribution?

- Trained on ImageNet, can ResNet50 classify every object in ImageNet, even when it is dark & distorted?

- If GPT6 can fit all published math papers —
  can GPT6 solve every problem that a mathematician can solve?

It is not 100% clear what “data distribution” we talk about.

The Meaning Changes Across Eras

Caveat 2: Self-supervised training.

Pretraining on next token prediction; inference on down-stream task.

What does “generalization” mean? **No clear definition.**

Caveat 3: Test-time compute.

The task is not to fit $y = f(x)$;

The output space has unbounded length. **More complicated.**

A core problem of ML —NOT the topic of this course.

SECTION 3B · DIAGNOSIS

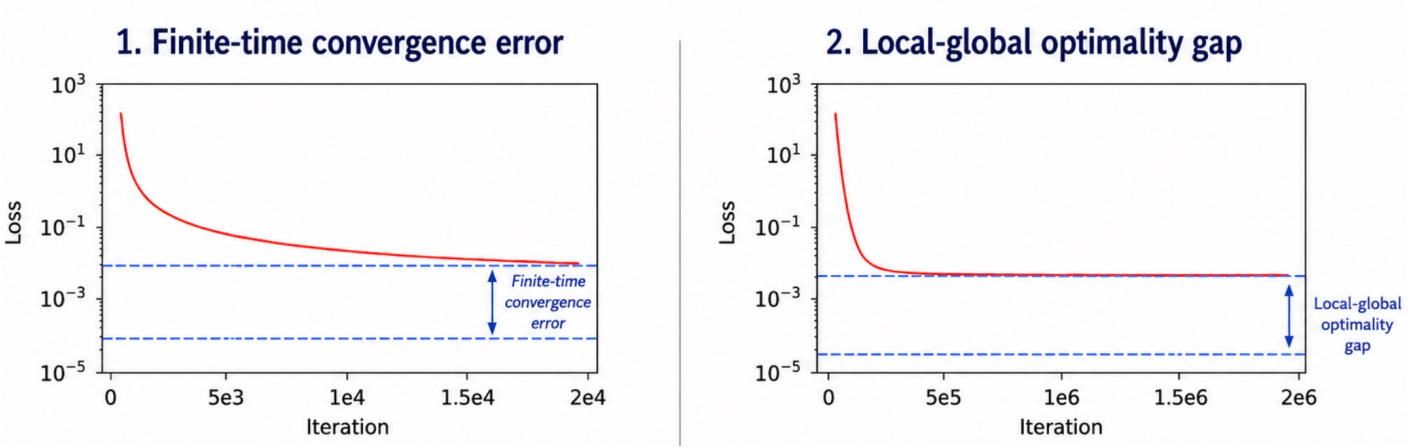
Optimization Error

not there yet, or stuck in a bad place?

Decomposition of Optimization

$$F(w_T) - F(w^*) = \underbrace{F(w_T) - F(\hat{w})}_{\text{finite-time convergence error}} + \underbrace{F(\hat{w}) - F(w^*)}_{\text{local-global optimality gap}}$$

w_T: iterate after T steps w^*: global minimum $\hat{w}$: stationary point approached by training



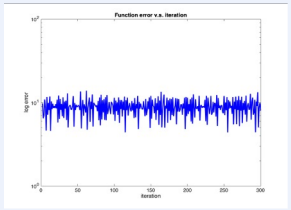
Two Simplifying Caveats

Caveat 1

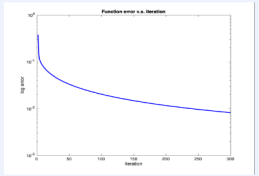
Finite-time convergence error

It may come from:

--non-convergence



--insufficient training



For simplicity, we can group these two effects together.

Caveat 2

"Local-global gap" is a simplification

Training may approach a stationary point, not necessarily a local minimum.

Strictly speaking, this part contains:
--stationary-to-local-min gap;
--local-to-global gap.

Closing "stationary-to-local-min" gap is related to **escaping saddle points.**
It is a rich area of study, but we omit for simplicity.

Hope 1: your algorithm finds global-min eventually

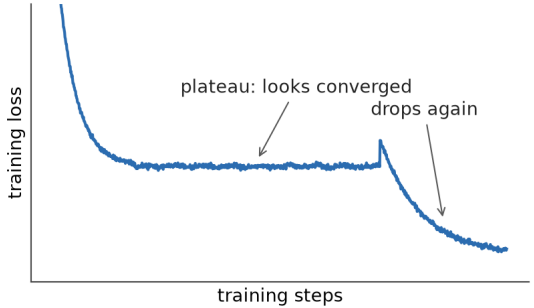
Note: This is about infinite-time behavior of the algorithm.

Not always true.

A central theme of research in the past decade.

Will discuss later.

Hope 2: Training Has Converged



How to tell? Flat loss curve?

Rigorously, NO. **Flat loss ≠ converged — may drop again.**

--Reason: "Flat loss curve" just means

$|F(w_{r+1}) - F(w_r)|$ is small (ST2);

Even if $|F(w_{r+1}) - F(w_r)| = 0$, it does not mean

Hope 2: Training Has Converged

By “training has converged”, an optimization :

the gradient is “small enough”.

（but how small is “small enough”? There is no concrete answer.）

Caveat 1: Implicitly, by “converged”, we mean: loss will not drop much even training 1000X time longer. Thus, it may be tricky to rigorously tell “training has converged”.

For practice, we can take “$\|\nabla F\| < \varepsilon$” where $\varepsilon = 10^{-3}, 10^{-6}, 10^{-8}$, etc., depending on experiences.

Caveat 2: Full gradient can be hard to compute.

In practice, as a compromise, can compute gradient for a subset of samples, or sometimes just regard “flat loss” as “converged”.

But be aware: this is an empirical compromise, not rigorous.

SECTION 3B · DIAGNOSIS

Representation Error

how do we know the representation power is not enough?

How Do We Know Representation Error?

In practice: given an architecture and a task —

how can we know whether it can fit the data?

e.g. for RNA prediction problem, will a 7B Qwen model be enough?

Well, if the training error can become nearly 0, surely it can fit the data.
But what if the training error is large?
How can we know it is due to architecture limitation, or algorithm limitation?

This decides which direction you should spend time (& money) on!

The Ideal Definition

Representation error = the **BEST** error the architecture can achieve.

[Disclaimer: for probability distribution estimation, we shall minus the absolutely
The smallest possible error; e.g. predicting a coin, the absolutely smallest error is 50%]

For “the best” to mean anything, two assumptions must hold:

Assumption 1: your algorithm will eventually converge to global-min

Assumption 2: your algorithm has converged

Computing Representation Error in Toy Experiments

For toy experiments:

fixed data, long enough training & many random initial points;

use this error as estimate of representation error of a given NN

[many random initial points: error similar, then less likely to be local minima;

long enough training: increasing iterations, error flat & gradient small]

--Not perfect validation of Assumption 1 & 2, but an empirical compromise.

Computing Representation Error Beyond Toy Experiments?

Only when BOTH assumptions hold, training error = representation error.

In practice, not that easy to find global optima.

If there is no guarantee that the algorithm has found nearly global minima, then there is no simple way to compute the representation error of a given NN.

A hard question.

There are some heuristics though… we skip for now.

SECTION · DEMO

Toy Experiments: Visualization of Loss and Three Errors

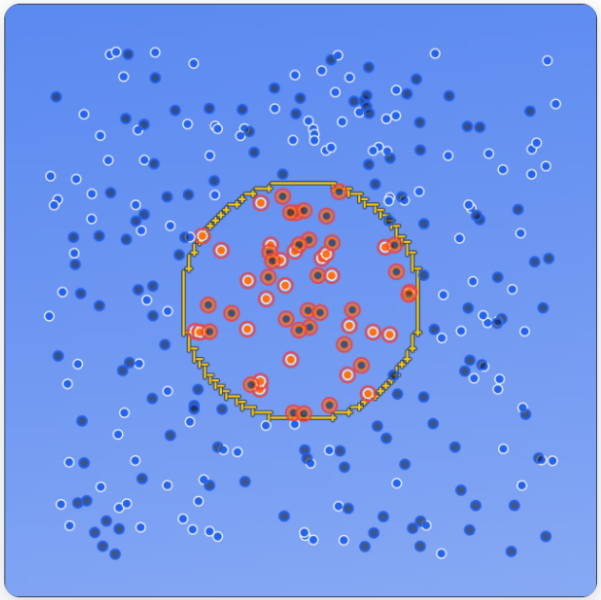
ruoyus.github.io/me2605/demos/nn-error-lab/

Three Errors

test loss ≈ **representation error** + **optimization error** + **generalization gap**

All bar values are estimates — for intuition, not proof.

Case 1: A Linear Net Cannot Learn a Circle



Live capture · decision boundary · epoch 4,000

PRESS Hidden layers → 0 (linear model). Play; let it run long.

YOU SEE train loss stuck at ≈ 0.44; representation bar full (0.4434); misclassified region = the whole circle.

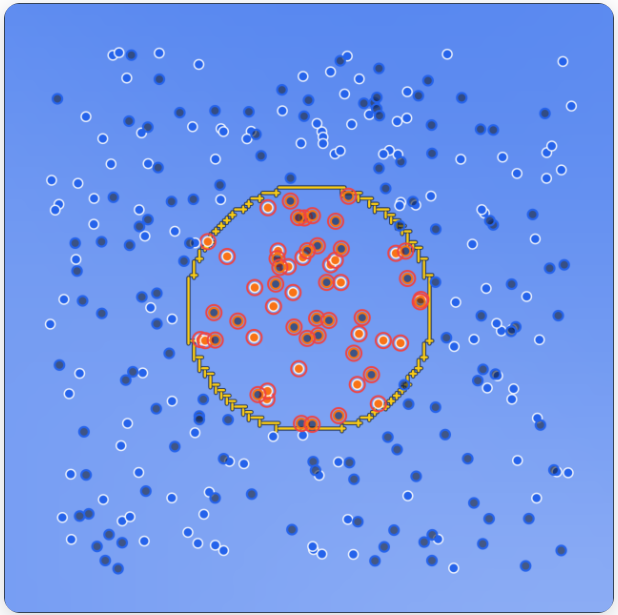
JUDGE No tuning can fix it — a structural ceiling. (Tick x_1^2, x_2^2 features and it learns instantly.)

The three bars · live estimate (full scale = 0.8)

Representation	0.4434
Optimization	0.0000
Generalization	0.0699

Representation error: the hypothesis class is too small.

Case 2: Learning Rate Too Small

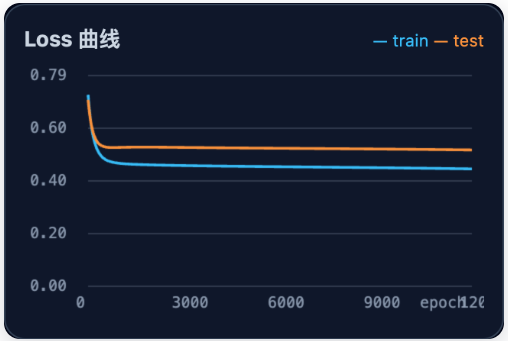


Live capture · decision boundary · epoch 12,000

PRESS 2×8 net; drag lr to 0.003. Play.

YOU SEE loss decreases very slowly — still ≈ 0.43 after 12k epochs; optimization bar stays large (0.4378).

JUDGE Structure is fine (representation ≈ 0) — SGD just has not gotten there. Time, or a larger lr, would fix it.

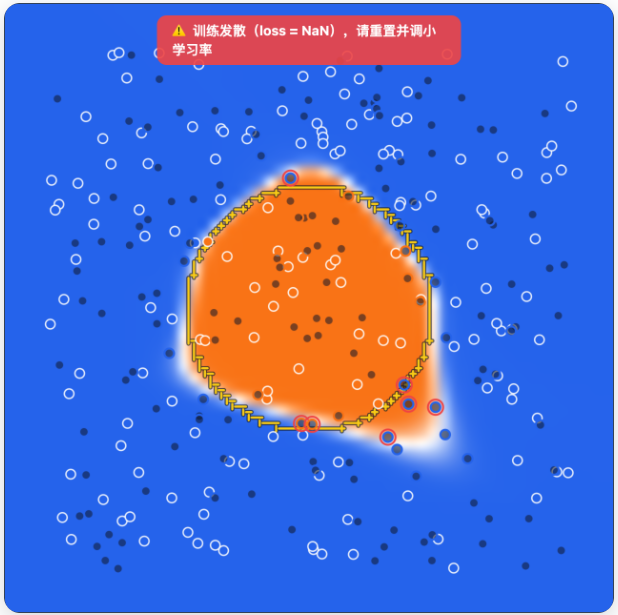


The three bars · live estimate (full scale = 0.8)

Representation	0.0000
Optimization	0.4378
Generalization	0.0714

Optimization error I: under-converged (too slow).

Case 3: Learning Rate Too Large

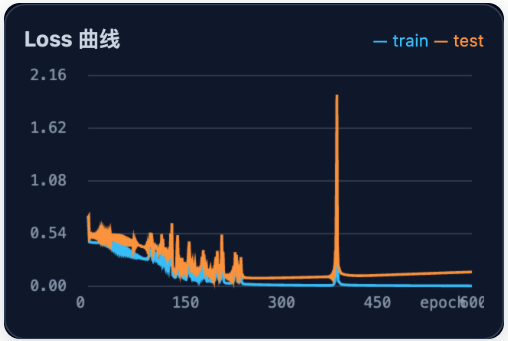


Live capture · decision boundary · epoch 600

PRESS Same 2×8 net; drag lr to 3. Play.

YOU SEE loss oscillates, then a single spike blows past 1.5 → NaN warning; training halts.

JUDGE Same error family as Case 2 — another way to die.



The three bars · live estimate (full scale = 0.8)

Representation

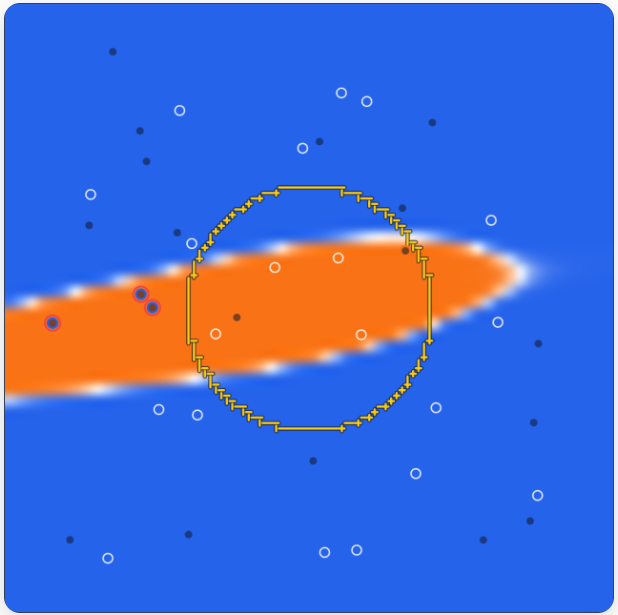
Optimization

Generalization

Bars unavailable — training diverged (loss blew past 1.5, NaN warning); the decomposition no longer applies.

Optimization error II: oscillation / divergence.

Case 4: Big Net, Few Data

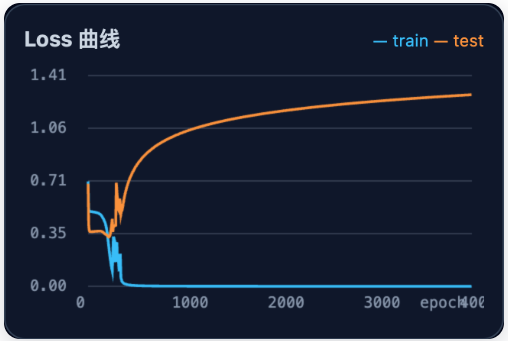


Live capture · decision boundary · epoch 4,000

PRESS 4×16 net; only 20 training samples. Play.

YOU SEE train loss → 0.00 while test loss climbs to ≈ 1.3; generalization bar overflows (1.2844); boundary bends around the 20 points and ignores the circle.

JUDGE The train/test gap — memorization, not learning.

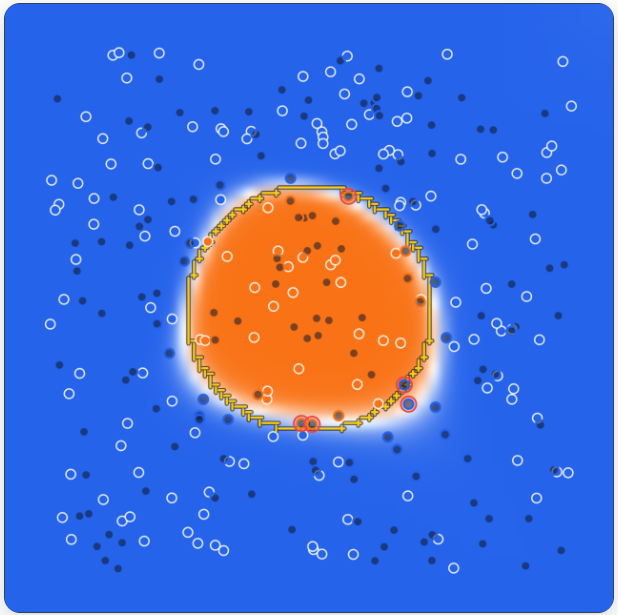


The three bars · live estimate (full scale = 0.8)

Representation	0.0000
Optimization	0.0000
Generalization	1.2844

Generalization error: train ≈ 0, test ≫ 0.

Case 5: Everything Just Right

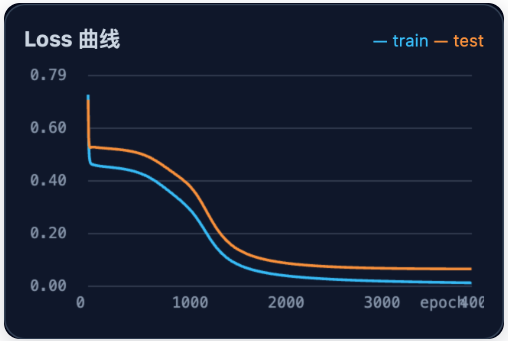


Live capture · decision boundary · epoch 4,000

PRESS 2×8 net, lr 0.1, 300 samples. Play.

YOU SEE all three bars small (0.0000 / 0.0081 / 0.0525); learned boundary hugs the true dashed circle; disagreement region ≈ 1.6%.

JUDGE The control group — this is the goal of tuning.



The three bars · live estimate (full scale = 0.8)

Representation	0.0000
Optimization	0.0081
Generalization	0.0525

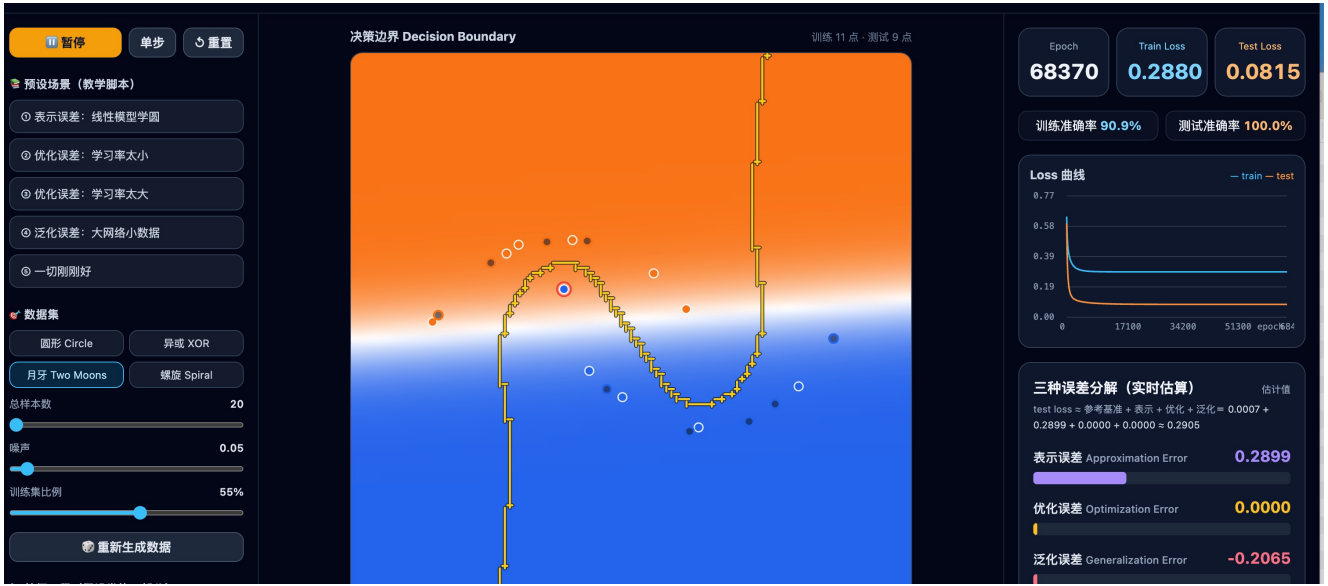
All three errors small: structure OK · optimizer OK · data OK.

Five Cases, One Diagnostic Table

#	SETUP	WHAT YOU SEE (MEASURED)	DOMINANT ERROR	FIRST FIX
1	0 hidden layers (linear), 300 samples	train loss stuck ≈ 0.44; whole circle misclassified	Representation 0.4434	add layers, or features x_1^2, x_2^2
2	2×8 net, lr 0.003	still ≈ 0.43 after 12k epochs	Optimization 0.4378	larger lr / train longer
3	2×8 net, lr 3	oscillation → spike → NaN warning	Optimization (diverged)	smaller lr
4	4×16 net, 20 samples	train 0.00 vs test ≈ 1.3 and rising	Generalization 1.2844	more data / smaller net / L2
5	2×8 net, lr 0.1, 300 samples	all bars ≈ 0; boundary hugs the circle	none — control	—

Diagnose before you tune: compare structures · check convergence · mind the train/test gap.

Strange Case: Negative Generalization Gap?



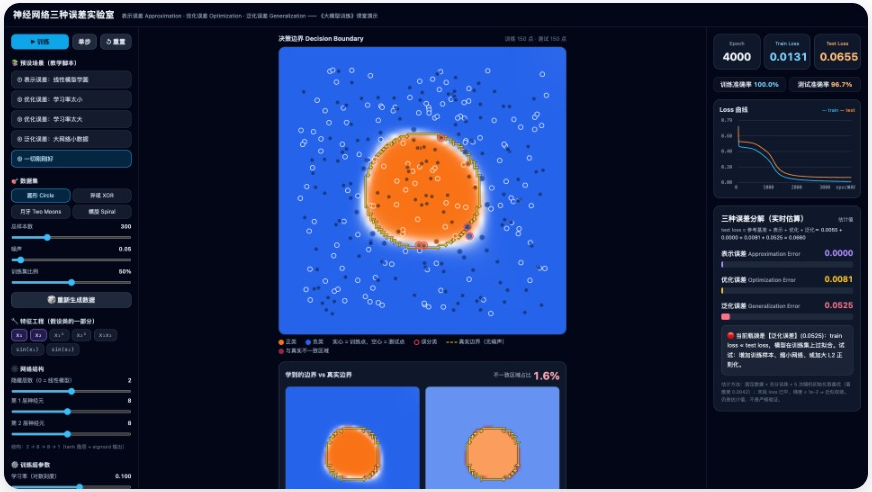
Why negative generalization gap? Tuning what, can fix it?

Answer: In this experiment, the test error is an inaccurate estimate of the true error on the whole distribution. To make it positive, can increase the total # of sample (总样本数).

Live: Your Turn to Call the Shots

ruoyus.github.io/me2605/demos/nn-error-lab/

- We re-run Cases 1–5 live.
- You call the parameters: **layers · lr · sample size**.
- **Predict first: which error bar will fill up?**
- Then try to break it: make *exactly one* bar big.



the lab: presets on the left match Cases 1–5; three bars on the right update live

Demo first, diagnosis second: read the bars before you touch the knobs.

References

Optimization for deep learning: theory and algorithms. Ruoyu Sun. (survey), arXiv.

Lectures on neural network optimization. Ruoyu Sun. (book draft).

QUIZ 2

The Three Errors

10 min · 15 True/False questions · pass ≥ 60 · covers Sections 2–3

Summary

- We discussed challenges & solutions in deep learning.
- High level: data + computation + model / algorithm.
- Mid level: representation + optimization + generalization.